COS341 2026: Continuation of **Semester-Practical-Specification** Part 3**b**) ***Dealing with Functions and Calls by way of "Inlining".***

General Idea:
All the code for functions gets generated "in between" **/* *C* comments */** and is, therefore, not executable. We will use this "code mine" for "mining" the function code for the "inlining" technique by **copying&pasting** the relevant code snippets into the executable MAIN code, as in the lecture. In "inlining", which works only for non-recursive functions (such as this year's version of the SPL), no runtime-stack is needed at all **:)**
Syntax/Grammar-based ***guidelines* for the translations are given below**, whereby the **green** code is the actual **resulting target code** which must actually get **written out** into the **output file**.
Note: these translation guidelines are giving you only the most important **hints**, but *the **details must be figured out by yourself***, at it is *your* project in which ***you*** must demonstrate ***your*** ingenuity!

SPL_PROG → P$ ***Hint:***

```
trans(SPL_PROG) = { trans(P) ASCII13
                    return 0; } /* end of executable C */
                  }
```

P → V_DECL **:** F_DECL **:** ALGO

Hint:

```
trans(P) = { /* begin of the non-executable code-mine for later inlining */ ASCII13
               trans(F_DECL) ASCII13
             /* end of the non-executable code-mine */ ASCII13
             /* begin of executable C code */ ASCII13
               #include <iostream> ASCII13
               int main() ASCII13
               { ASCII13
               trans(V_DECL) ASCII13
               trans(ALGO)
```

V_DECL → ε *// nullable* ***Hint:***
Translate the empty variable declaration into one blank space

---

V_DECL → **NAME** V_DECL

Hint:

```
trans(V_DECL) = { typ internalNewNameOf(NAME) ; ASCII13
                    trans(V_DECL)
                  }
```

Thereby:

- As in Part 3**a**, **typ** is either **int** (for the entire program), or **float** (for the entire program), depending on what the ***Type Analyser*** had previously found out in Phase 2**b**.
- To ***find out the internal new name*** of the declared variable's NAME, you must look into the ***Symbol Table*** from the semantic re-naming Phase 2**a**.

F_DECL → ε *// nullable* ***Hint:***
Translate the empty Function Declaration into one blank space

---

Turn the page ►

F_DECL → F_TYPE F_DECL

Hint:
trans(F_DECL) = { trans(F_TYPE) ASCII13
trans(F_DECL)
}

F_TYPE → **void NAME (V_DECL) { P return }**

Hint:
trans(F_TYPE) = { begin of code-mine for void-procedure *internalNewNameOf*(NAME) ASCII13
Parameters for inlining: ASCII13
trans(V_DECL) ASCII13
Body of the procedure: ASCII13
trans(P) ASCII13
end of code-mine for void-procedure *internalNewNameOf*(NAME) ASCII13
}

F_TYPE → **num NAME (V_DECL) { P return (TERM) }**

Hint:
Preparation:

- Let ***n*** be a freshly system-generated *unique new variable name*
- Let ***f*** be the *internalNewNameOf*(NAME) from the Symbol Table
- Let ++ be an auxiliary operation which appends two strings to each other
- Let ***r*** be ***f***++***n***

trans(F_TYPE) = { begin of code-mine for Function ***f*** ASCII13
Parameters for inlining: ASCII13
trans(V_DECL) ASCII13
Body of the procedure: ASCII13
trans(P) **;** ASCII13
trans(TERM , ***r***) ASCII13
end of code-mine for Function ***f*** ASCII13:
The result is found in the return value ***r*** ASCII13:
}

INPUT → ε *// nullable* ***Hint:*** *Translate the empty Input Parameter into one blank space*

Turn the page ►

TERM → CALL
INSTR → CALL
CALL → **NAME (INPUT)**

INPUT → TERM INPUT

Code Generation Advice:

- First, your code generator software need a while-loop that can count, in the syntax-tree, how many TERM-children the INPUT node has. Do this counting, and let **N** be the number of Non-empty TERM nodes which you have counted. *For example:* if the function is a ternary function, which takes three inputs, then **N=3**
- Next, you need a for-loop that iterates **N** times. In each of these iterations:
  - Let ***pN*** be a brand-new system-generated variable name, which is meant to "capture" the computational value of its corresponding input TERM.
  - Produce trans(TERM , ***pN***) **;** ASCII13

*Thereafter, you must produce **N** times, such as in the lecture:*

- *p1* = *a1* **;** ASCII13
- *p2* = *a2* **;** ASCII13
- ... etc ...
- *pN* = *aN* **;** ASCII13

whereby *a1, a2, ... , aN* are the input parameters from the V_DECL of the Function NAME in the non-executable Code-Mine, which has been created as described above.

Thereafter, as in the lecture, you **copy the generated body-code** of the function NAME ***from*** *the non-executable* ***Code-Mine*** **to here**.

Finally, as in the lecture, you take the return value r of that function, and you connect it with the continuation-variable of the caller, to conclude the inlining. If the function was a return-less void-Procedure, then this step is (of course) not needed. For example: Let ***c*** be the already existing continuation-variable of the caller-Algo, and let ***r*** be the return-variable of the callee-Function, then, as in the lecture, you must produce a code line ***c*** = ***r*** **;** ASCII13 to "finish" the inlining.

TO DO:

- **Finally implement the code generator for executable C code** *as outlined above.*
- ***Special care*** *must be taken in the following two* ***difficult scenarios****:*
  - If a function ***f*** calls its own sub-function ***g***, then the *1st phase of "Inlining" of* ***g*** *into* ***f*** takes place **inside the non-executable Code-Mine** before the so-produced ***f***-code finally gets copied out of the Code-Mine and "inlined" into the executable main Algo**!**
  - If the Body-code of a function ***f*** contains LABELS (as targets for GOTO Jumps), and if this function ***f*** gets called **more than once** from different positions, then with each copy-and-inline the LABELS must get uniquely re-named again! Otherwise the final target code **would *illegally* contain *duplicated* LABELS**, which would make the target code non-executable!
    EXAMPLE:
    a = 13
    x = call to Collatz(***a***) // ← Collatz-Code to be inlined *contains a while-**loop** with **labels***
    y = ***x*** + **1**
    z = call to Collatz(***y***) // ← 2nd inlining must **not** jump to the ***labels*** *of the 1st* inlining **!**
    print(***z***)

*As always: The hints of above do **not** tell you **exactly** how to solve every little detail of the problem. Thus:"Use your brain". If everything goes well then you should be able to run your compiled **SPL** Source Programs in your **C** environment!* And now: **HAPPY PROGRAMMING :)**